

PROJECT REPORT

Azure Honeypot with Microsoft Sentinel SIEM

Cloud-Based Threat Detection, Log Analytics & GeolP Attack Visualization

Submitted by

Lakkshanth R

B.Tech Computer Science and Engineering (Cyber Security)

Roohith R

B.Tech Computer Science and Engineering (Cyber Security)

Domain: Cloud Security | SIEM | Threat Detection

July 2026

Table of Contents

1. Executive Summary	3
2. Introduction	3
3. Project Objectives	4
4. Technologies and Tools Used	4
5. System Architecture	5
6. Project Methodology	6
6.1 Stage 1 — Azure Subscription Setup	6
6.2 Stage 2 — Honeypot Deployment	6
6.3 Stage 3 — Log Generation and Verification	6
6.4 Stage 4 — Centralized Log Collection and KQL	6
6.5 Stage 5 — GeoIP Enrichment	7
6.6 Stage 6 — Attack Map Visualization	7
7. Results and Observations	7
8. Key Windows Security Event IDs	8
9. Skills Applied and Learning Outcomes	8
9.1 Skills Applied	8
9.2 Learning Outcomes	9
10. Challenges and Limitations	9
11. Future Improvements	9
12. Conclusion	10
13. References	10

1. Executive Summary

This report presents the design, implementation, and analysis of a cloud-based Windows honeypot deployed on Microsoft Azure and monitored end-to-end through Microsoft Sentinel, Microsoft's cloud-native Security Information and Event Management (SIEM) platform. An internet-facing Windows 10 virtual machine was deliberately misconfigured — with a permissive Network Security Group and a disabled host firewall — to attract unsolicited, unauthenticated login attempts from across the public internet.

Security event logs generated on the honeypot were forwarded through the Azure Monitor Agent (AMA), governed by a Data Collection Rule (DCR), into a centralized Log Analytics Workspace (LAW). From there, events were queried using Kusto Query Language (KQL), enriched with geographic intelligence via a purpose-built GeoIP watchlist, and ultimately rendered on an interactive, JSON-configured attack map within a Microsoft Sentinel workbook.

The underlying objective of the project was to build hands-on, practitioner-level fluency with the core workflow used daily inside real-world Security Operations Centers (SOCs): log generation and verification, centralized log collection, query-driven investigation, threat-context enrichment, and executive-ready visualization. The project delivered a fully functioning, reproducible honeypot-to-dashboard pipeline capable of capturing failed logon attempts (Event ID 4625), resolving the geographic origin of attacking IP addresses with high fidelity, and plotting live attack activity on a global map — providing a compact but realistic analogue of production SOC tooling.

Beyond the technical build, the project reinforces transferable analytical skills: KQL query construction, watchlist-based data enrichment, and workbook-based visualization are conceptually portable to other SIEM platforms (e.g., Splunk, QRadar, Microsoft Defender), making the exercise directly relevant to future SOC analyst and cloud security engineering roles.

2. Introduction

Honeypots are decoy systems intentionally configured with weak, exposed, or otherwise misconfigured security controls in order to attract, detect, and study unauthorized access attempts. Within a Security Operations context, honeypots serve as a low-risk, high-value mechanism for observing genuine attacker behavior, validating detection pipelines end-to-end, and practicing log analysis and threat investigation without placing production systems or data at risk.

Public cloud platforms make this kind of exercise both cheaper and more realistic than a traditional on-premises lab: a virtual machine can be exposed to the live internet within minutes, and the resulting telemetry reflects genuine, unsolicited scanning and brute-force activity rather than synthetic or simulated traffic. This project deploys a Windows 10 virtual machine in Microsoft Azure as a honeypot, deliberately exposed to the public internet with permissive network rules and a disabled host firewall. The virtual machine's Windows Security Event Logs are centralized into an Azure Log Analytics Workspace

and analyzed using Microsoft Sentinel, enabling the team to detect brute-force and unauthorized login attempts, enrich them with geographic origin data, and visualize global attack activity in near real time.

The remainder of this report is organized as follows: Section 3 outlines the project's objectives; Section 4 lists the technologies and tools used; Section 5 describes the end-to-end system architecture; Section 6 walks through the project methodology stage by stage; Sections 7 through 9 present results, relevant event references, and skills gained; Section 10 discusses limitations; Section 11 proposes future improvements; and Sections 12 and 13 present the conclusion and references, respectively.

3. Project Objectives

- Deploy a Windows 10 virtual machine in Microsoft Azure and configure it as an internet-facing honeypot.
- Expose the virtual machine to unauthenticated inbound traffic by configuring the Network Security Group (NSG) and disabling the Windows Firewall across all profiles.
- Capture and verify failed login attempts locally using Windows Event Viewer (Event ID 4625).
- Forward Windows Security Events to a centralized Azure Log Analytics Workspace using the Azure Monitor Agent (AMA) and an associated Data Collection Rule (DCR).
- Integrate Microsoft Sentinel for centralized security monitoring, querying, and investigation.
- Query and filter security events using Kusto Query Language (KQL).
- Enrich failed login events with geographic (GeoIP) intelligence using a Microsoft Sentinel Watchlist and the `ipv4_lookup()` plugin.
- Build an interactive, JSON-configured attack map workbook to visualize the geographic origin and frequency of attacks.
- Document a reproducible methodology suitable for extension to further threat-hunting and detection-engineering work.

4. Technologies and Tools Used

The project draws on a representative cross-section of the tooling used in modern cloud-native SOC environments, spanning infrastructure provisioning, log routing, query analysis, threat enrichment, and visualization. Table 1 summarizes the technologies and tools employed at each stage of the pipeline.

Category	Technology / Tool	Role in the Project
Cloud Platform	Microsoft Azure	Hosting environment for all project infrastructure
Honeypot Host	Azure Virtual Machine (Windows 10)	Internet-facing decoy system generating attacker telemetry

Category	Technology / Tool	Role in the Project
SIEM Platform	Microsoft Sentinel	Central platform for querying, enrichment, and visualization
Log Aggregation	Azure Log Analytics Workspace (LAW)	Central repository for ingested security events
Log Forwarding Agent	Azure Monitor Agent (AMA)	Forwards Windows Security Events from the VM to the LAW
Log Routing Configuration	Data Collection Rule (DCR)	Defines which events are collected and where they are routed
Network Control	Network Security Group (NSG)	Permissive inbound rule set enabling honeypot exposure
Query Language	Kusto Query Language (KQL)	Filtering, correlation, and enrichment of security events
Threat Enrichment	GeoIP Watchlist (CSV, ~54,000 records)	Maps attacker IP ranges to geographic coordinates
Visualization	Microsoft Sentinel Workbook (JSON-based Attack Map)	Renders enriched events as an interactive geographic map
Local Log Inspection	Windows Event Viewer	On-host verification of captured Security events

Table 1. Technologies and tools mapped to their role in the honeypot pipeline.

5. System Architecture

The project follows a linear, five-stage data pipeline in which the honeypot generates raw security events, which are then centralized, queried, enriched with geographic context, and finally visualized. This architecture mirrors — at a smaller scale — the ingestion-to-dashboard pipelines used in production SOC environments, and is summarized in Figure 1.



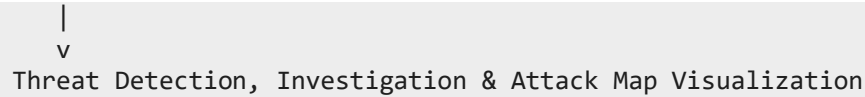


Figure 1. End-to-end log flow from the honeypot VM to Sentinel-based visualization.

Each stage in this pipeline maps directly onto a corresponding stage of the project methodology described in Section 6: VM exposure and log generation (Stages 1–3), centralized collection and querying (Stage 4), enrichment (Stage 5), and visualization (Stage 6). This alignment between architecture and methodology is intentional, and is designed to mirror the way SOC pipelines are typically documented and audited in industry.

6. Project Methodology

The project was executed in six sequential stages, moving from environment setup through to final attack visualization. Each stage builds directly on the artifacts produced by the previous one, so that the final attack map is traceable back to a specific, verifiable raw event.

6.1 Stage 1 — Azure Subscription Setup

An Azure subscription was created to provision the required cloud resources. Where a free-tier subscription was not available, a pay-as-you-go subscription — with diligent, scheduled resource shutdown to control cost — or an equivalent lab environment providing Azure, Microsoft Defender for Endpoint, and Tenable access was used instead. All environments were accessed through the Azure Portal (portal.azure.com), and resource groups were used throughout to keep the honeypot's infrastructure isolated and easy to tear down at the end of the exercise.

6.2 Stage 2 — Honeypot Deployment

A Windows 10 virtual machine was provisioned in Azure. The virtual machine was intentionally hardened in reverse — weakened, rather than secured — in order to attract attacker traffic:

- The Network Security Group (NSG) attached to the VM was modified to allow all inbound traffic, removing the default deny-by-default posture.
- The Windows Firewall was disabled on all profiles (Start → wf.msc → Properties → all profiles set to Off), removing the host-level layer of defense as well.
- Administrator credentials were recorded securely for later login, validation, and teardown, and were never reused on any production system.

This combination — permissive network policy plus a disabled host firewall — was deliberately chosen to maximize the probability of attracting internet-wide scanning and brute-force activity within a short observation window, while keeping the blast radius of the exercise fully contained to a disposable lab VM.

6.3 Stage 3 — Log Generation and Verification

To confirm that the honeypot correctly recorded unauthorized access attempts, three failed login attempts were performed using a generic username ("employee") to simulate a plausible brute-force target. After logging into the VM directly, Windows Event Viewer was used to inspect the Security log, where the three failed logon attempts were confirmed under Event ID 4625. This local verification step was treated as a control checkpoint — it confirmed that the VM itself was correctly generating and recording the events the remainder of the pipeline would depend on, before any dependency was placed on the cloud-side forwarding path.

6.4 Stage 4 — Centralized Log Collection and KQL

A Log Analytics Workspace (LAW) was created to serve as the central log repository, and a Microsoft Sentinel instance was connected to it. The "Windows Security Events via AMA" data connector was configured, which automatically provisions a Data Collection Rule (DCR) and installs the Azure Monitor Agent extension on the honeypot VM. Once log ingestion was confirmed, the following baseline KQL query was used to isolate failed logon events:

```
SecurityEvent  
| where EventID == 4625
```

This step validated that Windows Security Events were flowing correctly from the honeypot VM into the centralized workspace, and it established the core querying workflow — table selection, filtering, and time-window scoping — used throughout the remainder of the project.

6.5 Stage 5 — GeoIP Enrichment

Raw Security Events contain only the source IP address of a login attempt, with no inherent geographic context. To resolve attacker locations, a GeoIP watchlist (geoip-summarized.csv, approximately 54,000 rows mapping IP ranges to geographic data) was imported into Microsoft Sentinel as a Watchlist named "geoip", using "network" as the search key. Once imported, the watchlist was joined against the Security Events using the `ipv4_lookup()` plugin, enabling each failed login to be enriched with country, region, city, and coordinate data:

```
let GeoIPDB_FULL = _GetWatchlist("geoip");  
let WindowsEvents = SecurityEvent  
| where IPAddress == "<attacker IP address>"  
| where EventID == 4625  
| order by TimeGenerated desc  
| evaluate ipv4_lookup(GeoIPDB_FULL, IPAddress, network);  
WindowsEvents
```

In a production environment, this enrichment data would typically be sourced from a live, continuously updated GeoIP feed (for example, a commercial threat-intelligence API) rather than a static CSV import; the static watchlist approach was used here for simplicity, cost control, and reproducibility within a lab setting.

6.6 Stage 6 — Attack Map Visualization

A new Workbook was created within Microsoft Sentinel to visualize enriched attack data geographically. The default template elements were removed and replaced with a single Query element, configured through the Advanced Editor using a custom JSON definition (map.json). The underlying query reuses the GeoIP-enriched KQL logic from Stage 5, and the workbook's map visualization settings were configured to plot each failed login attempt as a point on a world map, sized and colored by frequency of occurrence — giving analysts an immediate, at-a-glance read on which regions were driving the bulk of observed attack volume.

7. Results and Observations

- The honeypot VM successfully attracted unauthenticated login attempts from external sources shortly after exposure, confirming the effectiveness of the permissive NSG rule and disabled firewall in generating genuine attacker traffic.
- Failed logon events (Event ID 4625) were reliably captured locally in Windows Event Viewer and, after AMA/DCR configuration, were reliably forwarded to the Log Analytics Workspace with no observed ingestion gaps.
- KQL queries against the SecurityEvent table allowed rapid filtering and investigation of failed login activity, demonstrating the practical value of a query language that shares strong conceptual overlap with SQL and Splunk's SPL.
- GeoIP enrichment via the Sentinel Watchlist successfully resolved attacker IP addresses to geographic locations, transforming raw, low-context IP data into actionable, geographically grounded threat intelligence.
- The Sentinel Workbook attack map provided a clear, visual representation of the geographic distribution of attack sources, making patterns in attacker origin — including concentration in specific regions — immediately apparent to a non-technical viewer as well as an analyst.
- Taken together, these results validate that a minimal, single-VM honeypot is sufficient to produce a realistic, end-to-end SOC-style detection and visualization pipeline within a short project timeline.

8. Key Windows Security Event IDs

Table 2 summarizes the Windows Security Event IDs most relevant to this project's monitoring scope. These IDs form the backbone of Windows-based logon auditing and are foundational reference points for any SOC analyst working with Windows telemetry.

Event ID	Description	Relevance to This Project
4625	An account failed to log on (failed logon attempt)	Primary event used to detect and enrich brute-force activity
4624	An account successfully logged on	Candidate for future detection of successful compromise

Event ID	Description	Relevance to This Project
4634	An account was logged off	Useful for session-duration and correlation analysis
4672	Special privileges assigned to a new logon	Indicator of privilege escalation for future hunting

Table 2. Key Windows Security Event IDs relevant to honeypot monitoring.

9. Skills Applied and Learning Outcomes

9.1 Skills Applied

- Cloud infrastructure deployment and administration on Microsoft Azure
- Configuration of Network Security Groups and host-based firewall controls
- Microsoft Sentinel SIEM administration and workbook design
- Centralized log management using Azure Log Analytics Workspace
- Query-based log analysis using Kusto Query Language (KQL)
- Threat intelligence enrichment using GeoIP watchlists and the `ipv4_lookup()` plugin
- Windows Security Event log analysis and investigation
- Data visualization and dashboard design for security operations

9.2 Learning Outcomes

This project reinforced the end-to-end workflow used by Security Operations Center (SOC) analysts on a daily basis: generating and verifying raw logs, centralizing them into a SIEM, querying and filtering them to isolate relevant events, enriching them with external threat context, and communicating findings through visualization. Notably, KQL — the query language used by Microsoft Sentinel — shares strong conceptual overlap with SQL and Splunk's SPL, meaning the querying skills developed here are directly transferable across SIEM platforms commonly used in industry, including Splunk and IBM QRadar.

10. Challenges and Limitations

- Scale: a single honeypot VM in one Azure region produces a limited sample of attacker traffic relative to a distributed, multi-region deployment, which may understate the true diversity of global attack sources.
- Static enrichment data: the GeoIP watchlist was a point-in-time CSV snapshot rather than a live feed, so IP-to-geography mappings may drift over time as address allocations change.

- No automated response: the pipeline is detection- and visualization-focused; it does not yet include automated containment actions such as IP blocking or account lockout.
- Single log source: the project scope was limited to Windows Security Events; it does not yet incorporate network flow logs, Defender for Endpoint signals, or Linux host telemetry, which would provide a fuller picture of attacker behavior.
- Cost and lifecycle management: an intentionally exposed VM must be monitored and torn down promptly to avoid runaway compute costs or unintended follow-on compromise beyond the observation window.

11. Future Improvements

- Deploy multiple honeypot virtual machines across different Azure regions to compare regional attack patterns.
- Extend monitoring to Linux-based honeypot systems in addition to the existing Windows deployment.
- Configure custom Sentinel Analytics Rules to automatically generate incidents from suspicious login patterns.
- Integrate Microsoft Defender for Endpoint for extended detection and response (XDR) capability.
- Automate incident response actions (e.g., IP blocking) using Azure Logic Apps playbooks.
- Map observed attacker behavior to the MITRE ATT&CK framework for structured threat classification.
- Develop additional threat-hunting dashboards covering successful logons (4624) and privilege escalation (4672).
- Replace the static GeoIP CSV with a continuously updated threat-intelligence feed to improve enrichment accuracy over time.

12. Conclusion

This project successfully demonstrated the deployment of a cloud-hosted Windows honeypot integrated with Microsoft Sentinel for centralized detection, investigation, and visualization of unauthorized access attempts. By progressing through virtual machine deployment, log generation, centralized log forwarding, KQL-based querying, GeoIP enrichment, and attack map visualization, the project provided practical, transferable experience with the core skill set required in modern Security Operations roles. The resulting pipeline offers a reusable foundation for further threat-hunting, detection engineering, and automation work in cloud-based security environments, and the stage-by-stage methodology documented here is intended to make that extension straightforward for future iterations of the project.

13. References

- Microsoft Azure Documentation — <https://learn.microsoft.com/en-us/azure/>
- Microsoft Sentinel Documentation — <https://learn.microsoft.com/en-us/azure/sentinel/>
- Azure Monitor Documentation — <https://learn.microsoft.com/en-us/azure/azure-monitor/>
- Kusto Query Language (KQL) Documentation — <https://learn.microsoft.com/en-us/kusto/query/>
- GeoIP Watchlist Data Source — <https://github.com/joshmadakor1/lognpacific-public>
- KC7 Cyber — Free KQL Practice Environment — <https://kc7cyber.com/>